

RNNCell#

<https://pytorch.org/docs/stable/generated/torch.nn.quantized.dynamic.RNNCell.html>

Description:

An Elman RNN cell with tanh or ReLU non-linearity. A dynamic quantized RNNCell module with floating point tensor as inputs and outputs. Weights are quantized to 8 bits. We adopt the same interface as `torch.nn.RNNCell`, please see <https://pytorch.org/docs/stable/nn.html#torch.nn.RNNCell> for documentation.

Function Signature

```
class torch.nn.quantized.dynamic.RNNCell(input_size,
hidden_size, bias=True, nonlinearity='tanh',
dtype=torch.qint8)[source]#
```

Code Examples

```
>>> rnn = nn.RNNCell(10, 20)
>>> input = torch.randn(6, 3, 10)
>>> hx = torch.randn(3, 20)
>>> output = []
>>> for i in range(6):
...     hx = rnn(input[i], hx)
...     output.append(hx)
```

Detailed Documentation

Documentation

An Elman RNN cell with tanh or ReLU non-linearity. A dynamic quantized RNNCell module with floating point tensor as inputs and outputs. Weights are quantized to 8 bits. We adopt the same interface as `torch.nn.RNNCell`, please see <https://pytorch.org/docs/stable/nn.html#torch.nn.RNNCell> for documentation.

An Elman RNN cell with tanh or ReLU non-linearity. A dynamic quantized RNNCell module with floating point tensor as inputs and outputs. Weights are quantized to 8 bits. We adopt the same interface as `torch.nn.RNNCell`, please see <https://pytorch.org/docs/stable/nn.html#torch.nn.RNNCell> for documentation.